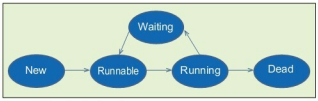


Figure 1: Life cycle of a thread

object in Java by extending the thread class of the *java.lang package.* For example:

```
class MyThread extends Thread{
public void run(){// line 2
System.out.println("Thread run method");// line 3

}
public static void main(String[] args){

     MyThread t = new MyThread();// line 6
     t.start();// line 7

}
}
```

In the above code, at line number 6, we have created a thread object. The *run()* method in the above code is a pre-defined method and it is present inside the *java.lang.Thread* class. In line number 7, when we say *t.sart()*, it actually calls the *run()* method and hence the output of the program is *"Thread run method"* (line number 3).

**Implementing the Runnable interface:**

```
class MyRunnable implements Runnable{

     public void run(){// line 2
       System.out.println("Runnable run method");// line 3
     }

     public static void main(String [] args){

       MyRunnable r = new MyRunnable();// line 6
       Thread t = new Thread(r);// line 7
       t.start();// line 8
     }
}
```

In the above code, you can see how it differs from extending a thread class. Here, at line number 6, we have created an object of the *MyRunnable* class, which is not a thread object but just an object of a class that implements the Runnable interface; hence, the object becomes Runnable. Now, since the object 'r' is Runnable, we can pass this object to a thread class constructor at line number 7 and, finally, once the thread object is created at line number 7, we call the *start()* method to start the thread that eventually gives the call to the *run()* method. And the output is *"Runnable run method"* (line 3).

***Note:*** We prefer implementing the Runnable interface over extending the thread class for thread implementation in Java, because the interface promotes multiple inheritance in Java.

## Life cycle of a thread

Let us go over the above thread's life cycle diagram (Figure 1), step by step:

1. *New:* The thread reaches this when we create the object of the thread. A thread is not considered alive at this state.
2. *Runnable:* Runnable means the thread is eligible to run but until the scheduler selects the thread to run, it can't run. A thread enters into this state when the *start()* method is invoked. A thread can also enter into this stage after running and in the blocked state. A thread is considered alive in this state.
3. *Running:* This is the state where the thread is actually performing the task. The thread comes into this state when the CPU scheduler selects the thread and makes it run. There are several ways to get to the runnable state, but there is only one way to get into the running state.
4. *Waiting/Blocked/Sleeping:* In these states, the thread is not eligible to run but it is still alive. From this state, the thread may return to the runnable state and further, it may go to the running state. The thread comes into this state when we call the *sleep(),wait()* and *yield()* methods.
5. *Dead:* A thread comes in this state when the *run()* method finishes its execution. Once a thread is dead, it can't be started again; so if you try to invoke the *start()* method again, it will give you an exception.

## Naming a thread

There are various ways to name a Java thread.

1. By using the thread class constructor:

```
public Thread(String)
public Thread(Runnable, String)

class MyThread extends Thread{

     public void run(){
       System.out.println(Thread.currentThread().getName());
     }

     public MyThread(String name){
       super(name);// line 6
     }
```